

Lessons from Mitigating False Positives in Google’s OSS-Fuzz-Gen

Paschal C. Amusuo
Purdue University
USA
pamusuo@purdue.edu

Jonathan Metzman
Google LLC
USA
metzman@google.com

Dongge Liu
Google LLC
Australia
donggelu@google.com

Oliver Chang
Google LLC
Australia
ochang@google.com

Ricardo Calvo
Purdue University
USA
rcalvome@purdue.edu

James C. Davis
Purdue University
USA
davisjam@purdue.edu

Abstract

Fuzz testing is widely used in industry and open-source ecosystems, but its effectiveness depends on the quality of fuzz drivers that translate raw fuzzer input into valid function arguments. Automated fuzz driver generation reduces manual effort, but in practice often produces false positive crashes, especially for complex functions. We introduce OSS-Fuzz-Gen, an experimental system for agent-based fuzz driver generation. This paper reports our experience mitigating its false positives using two complementary strategies: constraint-based fuzz driver generation and context-aware crash validation against realistic program entry points. In our evaluation on 1,500 benchmark functions from OSS-Fuzz, these strategies filtered out more than half of crashes initially classified as program errors, at modest additional cost. We distill lessons for deploying LLM-based agents in large-scale fuzzing pipelines.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; • **Security and privacy** → **Vulnerability scanners**.

Keywords

Fuzzing, False positives, LLMs, Agentic systems, Experience report

ACM Reference Format:

Paschal C. Amusuo, Dongge Liu, Ricardo Calvo, Jonathan Metzman, Oliver Chang, and James C. Davis. 2026. Lessons from Mitigating False Positives in Google’s OSS-Fuzz-Gen. In *34th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE Companion ’26)*, July 5–9, 2026, Montreal, QC, Canada. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3803437.3805205>

1 Introduction

Fuzz testing [28, 30] is a widely adopted technique for discovering bugs and security vulnerabilities in both commercial [29, 37] and open-source software [2, 12, 35]. In practice, effective fuzzing depends on the availability of high-quality fuzz drivers, which translate raw fuzzer input into the structured arguments and program state expected by target functions. Manually developing such

drivers is labor-intensive, requires project-specific expertise, and frequently limits coverage even in continuously fuzzed projects [4].

To address this bottleneck, both research and industry have explored automated fuzz driver generation using program analysis [14, 21, 36, 39] and large language models (LLMs) [23, 27, 38]. Google’s OSS-Fuzz-Gen project [23] applies LLM-based fuzz driver generation at scale across thousands of critical open-source projects. However, in operational use, this exposes a persistent challenge: false positive crashes caused by drivers that violate implicit input or state assumptions. As prior work observes [26, 31], this issue is intrinsic to function-level fuzzing — where regular functions are directly exercised without correct input validation and state initialization normally provided by higher-level callers. Existing mitigations struggle in OSS-Fuzz-Gen scale because they are either tightly coupled to specific program-analysis pipelines [26, 31] or analyze isolated crash locations [38] without whole-program usage context. At industrial scale, such false positives increase triage effort, delay real fixes, and undermine maintainer trust.

This paper reports on our experience mitigating false positive crashes in OSS-Fuzz-Gen through two complementary strategies deployed in a production fuzzing pipeline. Together, they provide the first whole-program-context-aware, agent-driven approach to reducing false positive crashes during fuzz driver generation. The first, *constraint-based fuzz driver generation*, proactively guides driver construction by enforcing inferred constraints on function inputs and state. The second, *context-aware crash validation*, reactively analyzes reported crashes to determine whether they are feasible from real program entry points. These strategies balance crash reduction with bug-finding effectiveness and operational cost.

We applied OSS-Fuzz-Gen with these strategies to 1,555 benchmark functions from OSS-Fuzz. Our strategies reduced the number of crashes by up to 8%, and eliminated up to 64% of crashes that would otherwise have been reported to maintainers, with modest additional cost relative to existing automation. Beyond these outcomes, our experience illustrates practical trade-offs between proactive and reactive false positive crash mitigation strategies and provides lessons and guidance to organizations integrating large language models in large-scale fuzzing workflows.

In summary, this paper makes the following contributions:

- An industry experience report on false positive crashes encountered during large-scale, automated fuzz driver generation.
- Two complementary mitigation strategies, integrated into a production fuzzing pipeline, that reduce false positives while preserving coverage and scalability.



This work is licensed under a Creative Commons Attribution 4.0 International License. FSE Companion ’26, Montreal, QC, Canada
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2636-1/26/07
<https://doi.org/10.1145/3803437.3805205>

- Lessons learned on the reliability, cost, and trust implications of deploying LLM-based agents in automated testing infrastructure.

2 Industrial Context

This section describes the operational setting in which our experience takes place. We summarize OSS-Fuzz as a large-scale continuous fuzzing service in §2.1, and then introduce OSS-Fuzz-Gen and its function-level fuzzing model in §2.2.

2.1 OSS-Fuzz and Large-Scale Fuzzing

OSS-Fuzz [1] is a long-running, production fuzzing service that provides continuous, large-scale fuzz testing for thousands of critical open-source projects [3]. It integrates coverage-guided fuzzers with automated build, execution, and crash reporting infrastructure, enabling sustained fuzzing across diverse languages, build systems, and project sizes without requiring maintainers to operate their own fuzzing environments. At this scale, a core goal of OSS-Fuzz is to surface *true*, actionable crashes for project maintainers to repair. False positives add triage cost and erode trust, as maintainers must repeatedly reproduce, inspect, and reason about reported failures. OSS-Fuzz has been the subject of much research [15, 16, 19, 32, 41].

OSS-Fuzz uses fuzz drivers to exercise specific functions or subsystems within a project. The performance of fuzzing depends heavily on the quality of these drivers [16]. Developing effective fuzz drivers often requires substantial engineering effort and deep project-specific knowledge. To encourage such investments, Google’s OSS-Fuzz Reward Program offers monetary rewards for integrations that exceed 50% coverage and uncover new bugs [5]. Yet, despite these incentives, many OSS-Fuzz projects still exhibit relatively low average coverage (around 30% [25]), emphasizing how difficult fuzz-driver development remains in practice. This persistent gap motivates automated fuzz-driver generation as a way to expand fuzzing reach while reducing maintainer and operator burden.

2.2 OSS-Fuzz-Gen: Generating Fuzz Harnesses

OSS-Fuzz-Gen [23] is an experimental project to automatically create fuzz drivers for OSS-Fuzz projects. Rather than relying on manually written harnesses for high-level entry points, OSS-Fuzz-Gen targets creating fuzz drivers for functions with low or no existing coverage. This function-level fuzzing model exercises internal library functions and is intended to expand coverage beyond what is practical to achieve through manual harness development alone.

OSS-Fuzz-Gen is designed as a multi-agent system that automates fuzz driver generation and refinement to maximize a project’s fuzzing coverage. Four distinct agents (1) write, (2) execute, (3) analyze fuzz drivers, and (4) use execution feedback to iteratively improve coverage. Figure 1 illustrates this architecture and how crashes and coverage results are used to guide refinement.

This expanded reach, however, introduces fundamental challenges. Many internal functions assume that inputs have been validated or that specific program state has been established by higher-level callers. Automatically generated drivers that invoke these functions in isolation may violate such assumptions, producing crashes that are infeasible in real executions. As OSS-Fuzz-Gen scales to thousands of functions, managing false positives while preserving coverage benefits is a central operational challenge.

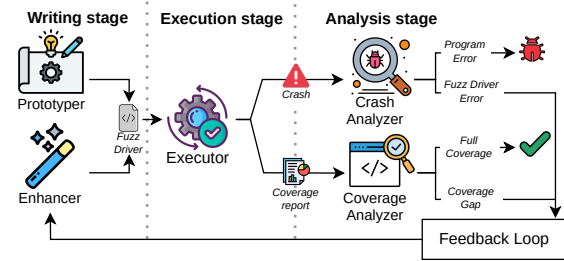


Figure 1: OSS-Fuzz-Gen design showing its agents. A function-level approach is taken, targeting functions with low coverage. This design exhibits a high false positive rate.

3 Problem: False Positive Crashes in Practice

This section describes the false positive crashes encountered in practice by OSS-Fuzz-Gen §3.1, and their effect (§3.2).

3.1 Sources of False Positive Crashes

During automated fuzz driver generation, many reported crashes arise from infeasible executions rather than defects that are reachable in real program use [26, 31, 38]. These false positives are especially common when (i) library APIs are called without the required preceding sequence of setup calls, or (ii) internal library functions are invoked directly, bypassing validation by higher-level callers.

Many functions targeted by function-level fuzzing assume that inputs have already been validated or correctly constructed. Functions may also rely on global state, object invariants, or environmental conditions that are implicitly satisfied along normal call paths or sequences but absent when invoked in isolation. Automatically generated fuzz drivers may fail to reconstruct these assumptions. The resulting crashes reflect mismatches between the fuzz driver and the function’s intended usage, not defects.

3.2 Impact on Maintainer Trust and Workflow

At industrial scale, false positive crashes impose costs that extend beyond wasted computation [6, 18]. Each reported crash requires triage effort to determine whether it represents a real defect or an artifact of the fuzzing setup, often involving reproduction, inspection of stack traces, and reasoning about infeasible execution paths. When such crashes are frequent, this triage burden can dominate the maintainer’s interaction with the fuzzing system.

Over time, repeated non-actionable crash reports erode trust in the fuzzing infrastructure itself [7, 22]. Maintainers may begin to deprioritize or ignore incoming reports, reducing the effectiveness of continuous fuzzing efforts. In shared infrastructure such as OSS-Fuzz, where credibility depends on consistently delivering high-signal findings across many projects, this erosion of trust represents a systemic failure mode rather than an isolated inconvenience.

4 Mitigation Strategies

To address the false positive crashes described in §3, we designed two complementary, agent-driven mitigation strategies that intervene at different stages of the OSS-Fuzz-Gen workflow. The first

strategy acts *proactively*, guiding fuzz driver generation to better respect a function's implicit input and state assumptions. The second strategy acts *reactively*, validating reported crashes against whole-program execution context to filter those that are infeasible in real executions. Together, these strategies reduce false positives while preserving the coverage benefits of function-level fuzzing.

Figure 2 illustrates how these strategies extend the baseline OSS-Fuzz-Gen pipeline introduced in Figure 1. A Constraint Generator agent derives semantic constraints from the target function and its surrounding context to guide fuzz driver construction. A Crash Validation agent analyzes crashes in the context of the project's entry points to distinguish true defects from artifacts of isolated invocation. Both strategies are implemented as LLM-driven agents equipped with access to the project codebase and analysis tools, and are integrated into the existing OSS-Fuzz-Gen feedback loop.

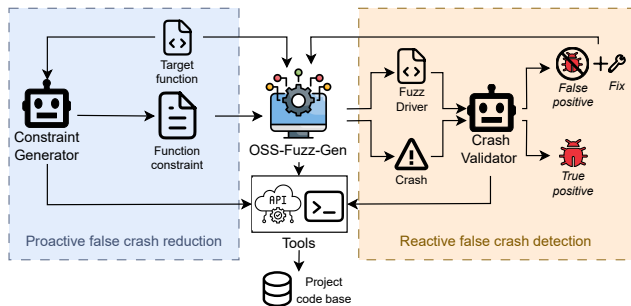


Figure 2: Agent-driven strategies to mitigate false positive crashes in OSS-Fuzz-Gen (cf. Figure 1). Semantic constraints developed by constraint generator improve fuzz driver quality and prevent false crashes. The crash validator analyzes project context to determine crash's feasibility and filter false positives. Agents use tools to access the project's codebase.

4.1 Constraint-Based Fuzz Driver Generation

Constraint-based fuzz driver generation targets the false positives that arise when drivers violate implicit assumptions about valid inputs or program state. As discussed in §3.1, many internal functions are designed to be called only after validation, normalization, or state initialization performed by higher-level callers. When such functions are fuzzed in isolation, unconstrained input generation frequently produces crashes that are infeasible in real executions. Rather than attempting to infer full correctness conditions, our approach focuses on recovering and enforcing a limited set of semantic constraints sufficient to avoid common infeasible executions.

Our approach uses a *Constraint Generator agent* to inspect the target function together with its call sites in the project codebase. Using this context, the agent identifies recurring assumptions about input structure, value ranges, object lifetimes, and state dependencies that are enforced by callers. These assumptions are translated into lightweight constraints that guide fuzz driver construction, such as restricting argument values, initializing required state, or respecting ordering constraints between operations. Like prior work [33], these constraints are not intended to be complete or sound; instead,

they serve as pragmatic guidance that steers fuzzing away from clearly infeasible inputs while leaving room for exploration.

Integration with OSS-Fuzz-Gen is deliberately lightweight. Derived constraints provide additional context to the fuzz driver generation and refinement agents, influencing how drivers are written and evolved across iterations. This design preserves the function-level nature of OSS-Fuzz-Gen and avoids tightly coupling constraint inference to any specific static analysis technique. We find that constraint-based generation reduces the volume of trivially infeasible crashes early in the pipeline, lowering downstream triage and validation effort with little false negatives.

4.2 Context-aware Crash Validation

While proactive constraints reduce many false positives, some infeasible crashes inevitably persist due to incomplete constraint recovery or complex execution conditions. context-aware crash validation addresses these remaining cases by analyzing reported crashes after execution to determine whether they are reachable from realistic program entry points. This shifts the question from whether a crash can occur under arbitrary invocation to whether it can occur within the project's intended usage.

The *Crash Validation agent* analyzes crashes in the context of the project's codebase. Starting from the crashing location, the agent examines how input arguments and relevant state are constructed along real call paths originating from entry points, such as public APIs or externally reachable interfaces. It then assesses whether the conditions required to trigger the crash can be satisfied under these call paths, or whether they depend on inputs or states that are never produced in practice. Crashes deemed infeasible are classified as false positives and filtered before reaching maintainers, while feasible crashes are retained for reporting and remediation.

Below is the core prompt for this agent. Crash details, code context, and tools are provided to support feasibility analysis.

Crash Validation Agent Prompt

You are a security engineer developing...Your goal is to analyze the crash details and determine if the crash is feasible from the project's entry points.

Input: [Input items...]

Steps to follow:

- Identify the crashing function and crash location.
- Determine the input conditions that caused the the crash.
- Identify how input arguments are created at call sites.
- Analyze whether constraints on input arguments could have prevented the crash.
- Provide your conclusion and code evidence for claims.

Output: [Output format...]

Tools provided: [Tool list...]

[Detailed Tool Instructions...]

4.3 A Note on Tools

Both mitigation strategies rely on the same underlying set of tools already used within OSS-Fuzz-Gen, including access to the project's source code, build artifacts, coverage reports, and crash diagnostics. Agents interact with these resources through programmatic interfaces for code search, compilation, and debugging, rather than through bespoke static or dynamic analyses. This design choice

Table 1: Crashes and coverage achieved by two OSS-Fuzz-Gen configurations (baseline/without function constraints, and new/with them). Parentheses denote number of false positive crashes. “# Bm” denotes number of benchmark functions.

Set	# Bm	Num. Crashes		% diff	Coverage	
		w/o FC	with FC		w/o FC	with FC
Set-1	510	1858 (1307)	1810 (1249)	2.6%	22.5%	22.3%
Set-2	510	1577 (1082)	1450 (1026)	8.1%	19.5%	19.0%
Set-3	535	1645 (1194)	1575 (1115)	4.3%	20.3%	20.1%
Total	1555	5080 (3583)	4835 (3390)	15.0%	62.3%	61.4%

reflects a deliberate emphasis on deployability: by reusing existing OSS-Fuzz infrastructure and lightweight tooling, the mitigation strategies can be integrated without introducing significant engineering overhead or specialized dependencies.

5 Deployment Experience

This section characterizes how these strategies behave in OSS-Fuzz-Gen. In summary, the Constraint Generator reduces crash rates by 2–8%, and the Crash Validation tool filters out ~50% of crashes.

5.1 Integration into OSS-Fuzz-Gen

We implemented the Function Analysis and Crash Validation agents using Google’s Agent Development Kit (ADK) and integrated them into OSS-Fuzz-Gen. OSS-Fuzz-Gen consists of 25KLoC of Python, spanning the core pipeline, agent implementations, support tooling (e.g., report generation), and experimental modules. Integrating the Constraint Generator and Crash Validation agents involved seven pull requests, adding 4,743 lines and removing 986. These changes introduced the new agents, corresponding prompts and updated OSS-Fuzz-Gen to support them.

5.2 Performance

Constraint Generator. Table 1 shows that incorporating function constraints reduced the total number of crashes by 2–8% across benchmark sets, with marginal changes in coverage. Measuring more deeply on 900 drivers, we found that with function constraints enabled, the share of drivers satisfying all constraints rose from 38.9% to 63.1%, and overall constraint satisfaction increased from 73.2% to 86.9%, with comparable constraint counts per driver. Proactive constraints improve fuzz driver quality without compromising code exploration, addressing our team’s concern that stricter driver generation would suppress bug-finding effectiveness.

Crash Validator. Per Table 2, the Crash Validation agent filtered out 57%–65% of crashes initially classified as “Program Errors,” reducing false positives that would otherwise require manual triage. For example, Listing 1 illustrates a case where the agent correctly deemed a crash infeasible: although triggerable in the target function, the program’s entry-point input validation prevents this crash.

The Crash Validator relies more on the agent’s ability to integrate tools, and hence poor reasoning is more likely. To assess the

Table 2: Percentage of crashes classified as “Program Errors” that were filtered out by the crash validator (§4.2).

Metric	Set 1	Set 2	Set 3
Crashes caused by “program errors”	1092	840	853
Additional false positives identified	626	548	522
% non-feasible crashes filtered	57.3%	65.2%	61.2%

```

1 <feasible> False </feasible>
2 <analysis>
3 The crash is a heap-buffer-overflow read in
  `flexbuffers::GetRoot` caused by reading from a buffer with a
  size < 2. The function reads the last two bytes of the buffer to
  determine the `byte_width` and `packed_type` without first
  checking if the buffer is large enough...The fuzzer triggers
  this...calling `flexbuffers::GetRoot` with a 1-byte buffer.
4
5 The crash is not feasible. When `flatc` processes a FlexBuffer
  file, it invokes `flexbuffers::VerifyBuffer` before calling
  `flexbuffers::GetRoot`. This verification step ensures the
  buffer is ≥ 3 bytes long, which is a stricter check than
  required to prevent the crash (a 2-byte minimum).
6 ...
7 </analysis>
8 <source_code_evidence>...</source_code_evidence>
9 <recommendations>...</recommendations>

```

Listing 1: Analysis from crash validation agent, showing why a specific crash is infeasible from the program’s entry point.**Table 3: Reliability assessment of crash analyzer.**

Metric	Count
Crashes reviewed	20
Analysis with reliable conclusions	11 (55%)
Crashes classified as false positives	6/10 (60%)
Crashes classified as true bugs	5/10 (50%)
Incorrect localization of crash	5 (25%)
Incomplete consideration of call sites	2 (10%)
Incorrect identification of root cause	1 (5%)
Incorrect constraint and feasibility analysis	1 (5%)

reliability of this component, we manually examined 20 of its analyses. 55% contained no identifiable flaws. We considered five of them as real defects and disclosed them to the project maintainers. The others were imprecise for a variety of reason (Table 3). Notably, 4 of 5 crash localization errors came from wrong root cause analysis produced by upstream agents.

Cost. Our approach eliminated 57–65% of crashes that would have been reported to maintainers. Given the time it takes engineers to triage, this translates to major savings in engineering hours. These benefits come with modest additional runtime costs. Table 4 summarizes the average per-driver cost of running OSS-Fuzz-Gen with the added agents. The Constraint Generator incurs minimal overhead, while the Crash Validation agent is more expensive due to deeper analysis and tool usage. Together, the two agents increase OSS-Fuzz-Gen’s API cost by approximately 9.3%, or an average cost of \$0.94 per generated driver and \$43.50 per evaluated project in our measurement. In the context of continuous fuzzing, where

Table 4: Average per-driver cost of OSS-Fuzz-Gen agents (USD). When applying these agents to generate and refine a driver for a single function within an OSS-Fuzz project, the new agents add roughly 9% to the cost. Averaging 10 concurrent trials per function and 1555 functions from 336 projects, executing OSS-Fuzz-Gen with the introduced agents cost \$43.50 per evaluated project and \$14,616 for all 336 projects.

Agent	Input	Tools	Output	Total
Constraint Generator	\$0.004	\$0.016	\$0.004	\$0.024
Crash Validator	\$0.022	\$0.023	\$0.012	\$0.056
Existing agents	\$0.055	\$0.685	\$0.119	\$0.859
Cost (per driver)	\$0.081	\$0.723	\$0.135	\$0.939
Cost (per project)	\$3.75	\$33.49	\$6.26	\$43.50

generated drivers are executed repeatedly over long periods, these one-time generation and validation costs are small relative to ongoing execution costs.

Analysis. The two strategies have complementary properties. Constraint-based generation reduces the volume of invalid drivers early, lowering downstream analysis and validation effort. Context-based validation, while more costly, substantially reduces maintainer-facing false positives and serves as a safeguard against incomplete or imprecise constraints.

6 Lessons Learned and Future Directions

This section distills lessons from deploying these techniques within OSS-Fuzz-Gen and outlines practical directions for researchers improving large-scale, automated fuzz driver generation and practitioners deploying them in other contexts.

Lesson 1: Constraints reduce noise, but are insufficient.

Constraint-based fuzz driver generation reduces surface-level false positives by preventing clearly invalid invocations, but its impact is inherently bounded. Mitigating all false crashes is intractable because many occurred in helper functions beyond the constraint generator’s scope or required more complex constraints. Thus, constraints are most effective as a noise-reduction mechanism to lower downstream validation cost, but are insufficient alone. These results also motivate new techniques to improve fuzz-driver quality through richer constraints without sacrificing bug discovery.

Lesson 2: Whole-program context improves crash triage.

The largest reduction in maintainer-facing false positives came from validating crashes in the context of realistic program entry points. Many crashes that appear plausible when a function is invoked in isolation are infeasible under normal execution. Filtering these cases substantially reduces triage effort and improves trust in automated reports, suggesting that whole-program context should be treated as a first-class concern when scaling function-level fuzzing.

Lesson 3: Agent reliability depends on upstream signal quality. The effectiveness of downstream validation agents in multi-agent pipelines is strongly shaped by the quality of upstream analyses. Incomplete or inaccurate root-cause information propagates through the pipeline and led agents to draw incorrect conclusions despite following prescribed reasoning steps. This highlights a

system-level challenge in multi-agent pipelines, where uncertainty compounds across stages and must be managed explicitly.

Lesson 4: Cost overhead is modest and manageable. The added cost of constraint inference and crash validation is small relative to the overall cost of continuous fuzzing. Driver generation and validation incur under one dollar per function on average and increase API usage by less than ten percent, while being amortized across long-running fuzzing campaigns. In practice, cost did not limit deployment, making these strategies viable at industrial scale.

Lesson 5: Progress may need tighter integration, not new techniques. OSS-Fuzz-Gen already improves coverage and finds real bugs, but false positives remain a challenge. Our experience suggests that further gains can come from tighter coordination between agents, richer shared context, and improved validation against realistic execution paths, rather than requiring fundamentally new mechanisms. Combining lightweight semantic guidance with context-aware validation appears to be a durable pattern for scaling automated testing while maintaining maintainer trust.

7 Related Work

Prior work has explored automated fuzz-driver generation using both program analysis [14, 21, 26, 31, 36] and AI-driven methods [23, 27, 38, 40]. False-positive crashes remain a recurring challenge across both lines of work. Program-analysis approaches mitigate them using static analysis methods to assess crash feasibility in the broader application [26, 31]. In contrast, existing LLM-based methods mitigate them by analyzing the crashing function in isolation [23, 38]. This localized method is insufficient in OSS-Fuzz-Gen because it ignores whether crash-triggering inputs are feasible under normal execution. This work addresses this limitation by combining semantic guidance during fuzz-driver generation with context-aware crash validation.

Other function-level analyses, such as underconstrained symbolic execution [20, 34] and unit proofing [9, 10, 13], reason more symbolically about function behavior and can provide stronger evidence about vulnerability presence or absence. However, imprecise environment models in these methods also introduce infeasible alarms. This suggests that the proactive constraint generation and reactive context-aware reasoning in this work may also mitigate false positives in these domains.

8 Conclusion

Our goal is to make automated fuzz driver generation more usable by reducing false positive crashes. In OSS-Fuzz-Gen, a function-level fuzzing strategy expands coverage but also produces many crashes that are infeasible in real executions, increasing triage costs. We reported on mitigating false positives by combining lightweight constraints during driver generation with context-aware crash validation against project entry points. We learned that constraints reduce noise but do not eliminate false positives, whole-program context substantially benefits crash triage, and the costs of an agentic approach are modest enough to support practical deployment.

Data Availability: The OSS-Fuzz-Gen repository [17] contains the implementation of Figures 1 and 2. Evaluation data are in [8]. Our blog [24] and technical report [11] provide illustrative examples, agent prompts, and further evaluation.

References

- [1] 2016. *Announcing OSS-Fuzz: Continuous fuzzing for open source software*. <https://opensource.googleblog.com/2016/12/announcing-oss-fuzz-continuous-fuzzing.html>
- [2] 2019. *A gentle introduction to Linux Kernel fuzzing*. <https://blog.cloudflare.com/a-gentle-introduction-to-linux-kernel-fuzzing/>
- [3] 2025. *Accepting new projects*. <https://google.github.io/oss-fuzz/getting-started/accepting-new-projects/>
- [4] 2025. *Fuzzing | Project Profile*. <https://introspector.oss-fuzz.com/>
- [5] 2025. *OSS-Fuzz Reward Program Rules | Google Bug Hunters*. <https://bughunters.google.com/about/rules/open-source/5097259337383936/oss-fuzz-reward-program-rules>
- [6] Bushra A. Alahmadi, Louise Axon, and Ivan Martinovic. 2022. 99% False Positives: A Qualitative Study of SOC Analysts' Perspectives on Security Alarms. 2783–2800. <https://www.usenix.org/conference/usenixsecurity22/presentation/alahmadi>
- [7] Amit Seal Ami, Kevin Moran, Denys Poshyvanyk, and Adwait Nadkarni. 2024. "False negative - that one is going to kill you": Understanding Industry Perspectives of Static Analysis based Security Testing. In *2024 IEEE Symposium on Security and Privacy (SP)* (2024-05). 3979–3997. doi:10.1109/SP54263.2024.00019 ISSN: 2375-1207.
- [8] Paschal C. Amusuo. 2026. *FSE-Industry-2026-OSSFuzzGen-FalseCrashReducer*. <https://doi.org/10.5281/zenodo.19392180>. GitHub repository.
- [9] Paschal C. Amusuo, Ricardo Calvo, Dharun Anandayavaraj, Taylor Le Lievre, Kevin Kolyakov, Elijah Jorgensen, Aravind Machiry, and James C. Davis. 2026. *AutoSOP: Safety-Oriented Unit Proof Generation for Component-level Memory-Safety Verification*. arXiv:2605.10712 [cs.SE] doi:10.48550/arXiv.2605.10712
- [10] Paschal C. Amusuo, Owen Cochell, Taylor Le Lievre, Parth V. Patil, Aravind Machiry, and James C. Davis. 2025. *Do Unit Proofs Work? An Empirical Study of Compositional Bounded Model Checking for Memory Safety Verification*. In *2026 IEEE/ACM 48th International Conference on Software Engineering*. arXiv:2503.13762 [cs] doi:10.48550/arXiv.2503.13762
- [11] Paschal C. Amusuo, Dongge Liu, Ricardo Andres Calvo Mendez, Jonathan Metzman, Oliver Chang, and James C. Davis. 2025. *FalseCrashReducer: Mitigating False Positive Crashes in OSS-Fuzz-Gen Using Agentic AI*. arXiv:2510.02185 [cs.SE] <https://arxiv.org/abs/2510.02185>
- [12] Paschal C. Amusuo, Ricardo Andrés Calvo Méndez, Zhongwei Xu, Aravind Machiry, and James C. Davis. 2023. *Systematically Detecting Packet Validation Vulnerabilities in Embedded Network Stacks*. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (2023-09). 926–938. doi:10.1109/ASE56229.2023.00095 ISSN: 2643-1572.
- [13] Paschal C. Amusuo, Parth V. Patil, Owen Cochell, Taylor Le Lievre, and James C. Davis. 2025. *A Unit Proofing Framework for Code-level Verification: A Research Agenda*. In *2025 IEEE/ACM 47th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*. 36–40. doi:10.1109/ICSE-NIER66352.2025.00013 ISSN: 2832-7632.
- [14] Domagoj Babić, Stefan Bucur, Yaohui Chen, Franjo Ivančić, Tim King, Markus Kusano, Caroline Lemieux, László Szekeres, and Wei Wang. 2019. *FUDGE: fuzz driver generation at scale*. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (New York, NY, USA) (ESEC/FSE 2019). Association for Computing Machinery, 975–985. doi:10.1145/3338906.3340456
- [15] Zhen Yu Ding and Claire Le Goues. 2021. *An Empirical Study of OSS-Fuzz Bugs*. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. 131–142. doi:10.1109/MSR52588.2021.00026 ISSN: 2574-3864.
- [16] Wentao Gao, Van-Thuan Pham, Dongge Liu, Oliver Chang, Toby Murray, and Benjamin I.P. Rubinstein. 2023. *Beyond the Coverage Plateau: A Comprehensive Study of Fuzz Blockers (Registered Report)*. In *Proceedings of the 2nd International Fuzzing Workshop* (New York, NY, USA) (FUZZING 2023). Association for Computing Machinery, 47–55. doi:10.1145/3605157.3605177
- [17] Google. 2024. *oss-fuzz-gen: LLM powered fuzzing via OSS-Fuzz*. <https://github.com/google/oss-fuzz-gen>. GitHub repository.
- [18] Zhaoqiang Guo, Tingting Tan, Shiran Liu, Xutong Liu, Wei Lai, Yibiao Yang, Yanhui Li, Lin Chen, Wei Dong, and Yuming Zhou. 2023. *Mitigating False Positive Static Analysis Warnings: Progress, Challenges, and Opportunities*. 49, 12 (2023), 5154–5188. doi:10.1109/TSE.2023.3329667
- [19] Philipp Götz, Joshua Schilling, Thorsten Holz, and Marcel Böhme. 2025. *An Empirical Study of Fuzz Harness Degradation*. arXiv:2505.06177 [cs] doi:10.48550/arXiv.2505.06177
- [20] Yu Hao, Juefei Pu, Xingyu Li, Zhiyun Qian, and Ardalan Amiri Sani. 2025. *SyzSpec: Specification Generation for Linux Kernel Fuzzing via Under-Constrained Symbolic Execution*. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security* (New York, NY, USA, 2025-11-22) (CCS '25). Association for Computing Machinery, 813–826. doi:10.1145/3719027.3744811
- [21] Kyriakos Ispoglou, Daniel Austin, Vishwath Mohan, and Mathias Payer. 2020. *FuzzGen: Automatic Fuzzer Generation*. 2271–2287. <https://www.usenix.org/conference/usenixsecurity20/presentation/ispoglou>
- [22] Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. 2013. *Why don't software developers use static analysis tools to find bugs?*. In *2013 35th International Conference on Software Engineering (ICSE)* (2013-05). 672–681. doi:10.1109/ICSE.2013.6606613 ISSN: 1558-1225.
- [23] Dongge Liu, Oliver Chang, Jonathan Metzman, Martin Sablotny, and Mihai Maruseac. 2024. *OSS-Fuzz-Gen: Automated Fuzz Target Generation*. <https://github.com/google/oss-fuzz-gen>
- [24] Dongge Liu, Jonathan Metzman, Oliver Chang, and Google Open Source Security Team. 2023. *AI-Powered Fuzzing: Breaking the Bug Hunting Barrier*. <https://security.googleblog.com/2023/08/ai-powered-fuzzing-breaking-bug-hunting.html>. Accessed January 22, 2026.
- [25] Dongge Liu, Jonathan Metzman, Oliver Chang, and Google Open Source Security Team. 2023. *AI-Powered Fuzzing: Breaking the Bug Hunting Barrier*. <https://security.googleblog.com/2023/08/ai-powered-fuzzing-breaking-bug-hunting.html>
- [26] Yuwei Liu, Yanhao Wang, Xiangkun Jia, Zheng Zhang, and Purui Su. 2024. *AFGen: Whole-Function Fuzzing for Applications and Libraries*. In *2024 IEEE Symposium on Security and Privacy (SP)*. 1901–1919. doi:10.1109/SP54263.2024.00011 ISSN: 2375-1207.
- [27] Yunlong Lyu, Yuxuan Xie, Peng Chen, and Hao Chen. 2024. *Prompt Fuzzing for Fuzz Driver Generation*. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security* (New York, NY, USA) (CCS '24). Association for Computing Machinery, 3793–3807. doi:10.1145/3658644.3670396
- [28] Valentin J.M. Manès, HyungSeok Han, Choongwoo Han, Sang Kil Cha, Manuel Egele, Edward J. Schwartz, and Maverick Woo. 2021. *The Art, Science, and Engineering of Fuzzing: A Survey*. *IEEE Transactions on Software Engineering* 47, 11 (11 2021), 2312–2331. doi:10.1109/TSE.2019.2946563 Conference Name: IEEE Transactions on Software Engineering.
- [29] Paul Marinescu. 2021. *Autonomous testing of services at scale*. <https://engineering.fb.com/2021/10/20/developer-tools/autonomous-testing/>
- [30] Barton P. Miller, Lars Fredriksen, and Bryan So. 1990. *An empirical study of the reliability of UNIX utilities*. *Commun. ACM* 33, 12 (01 12 1990), 32–44. doi:10.1145/96267.96279
- [31] Siddharth Muralee, Sourag Cherupattamoolayil, James C. Davis, Antonio Bianchi, and Aravind Machiry. 2025. *Reactive Bottom-Up Testing*. arXiv:2509.03711 [cs] doi:10.48550/arXiv.2509.03711
- [32] Olivier Nourry, Yutaro Kashiwa, Weiye Shang, Honglin Shu, and Yasutaka Kamei. 2025. *My Fuzzers Won't Build: An Empirical Study of Fuzzing Build Failures*. *ACM Trans. Softw. Eng. Methodol.* 34, 2 (20 01 2025), 29:1–29:30. doi:10.1145/3688842
- [33] Joonyoung Park, Inho Lim, and Sukyoung Ryu. 2016. *Battles with false positives in static analysis of JavaScript web applications in the wild*. In *Proceedings of the 38th International Conference on Software Engineering Companion* (New York, NY, USA, 2016-05-14) (ICSE '16). Association for Computing Machinery, 61–70. doi:10.1145/2889160.2889227
- [34] David A. Ramos and Dawson Engler. 2015. *{Under-Constrained} Symbolic Execution: Correctness Checking for Real Code*. 49–64. <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/ramos>
- [35] Kostya Serebryany. 2017. *{OSS-Fuzz} - Google's continuous fuzzing service for open source software*. (2017). <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/serebryany>
- [36] Gabriel Sherman and Stefan Nagy. 2025. *No Harness, No Problem: Oracle-guided Harnessing for Auto-generating C API Fuzzing Harnesses*. *IEEE Computer Society*, 165–177. doi:10.1109/ICSE55347.2025.00239
- [37] Tyson Smith, Jesse Schwartzentruber, and Sylvestre Ledru. 2021. *Browser fuzzing at Mozilla*. <https://blog.mozilla.org/attack-and-defense/2021/05/20/browser-fuzzing-at-mozilla>
- [38] Hanxiang Xu, Wei Ma, Ting Zhou, Yanjie Zhao, Kai Chen, Qiang Hu, Yang Liu, and Haoyu Wang. 2025. *CKGFuzzer: LLM-Based Fuzz Driver Generation Enhanced By Code Knowledge Graph*. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. 243–254. doi:10.1109/ICSE-Companion66252.2025.00079 ISSN: 2574-1934.
- [39] Cen Zhang, Xingwei Lin, Yuekang Li, Yinxing Xue, Jundong Xie, Hongxu Chen, Xinlei Ying, Jiashui Wang, and Yang Liu. 2021. *APICraft: Fuzz Driver Generation for Closed-source SDK Libraries*. 2811–2828. <https://www.usenix.org/conference/usenixsecurity21/presentation/zhang-cen>
- [40] Cen Zhang, Yaowen Zheng, Mingqiang Bai, Yeting Li, Wei Ma, Xiaofei Xie, Yuekang Li, Limin Sun, and Yang Liu. 2024. *How Effective Are They? Exploring Large Language Model Based Fuzz Driver Generation*. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (New York, NY, USA) (ISSTA 2024). Association for Computing Machinery, 1223–1235. doi:10.1145/3650212.3680355
- [41] Yuntong Zhang, Jiawei Wang, Dominic Berzin, Martin Mirchev, Dongge Liu, Abhishek Arya, Oliver Chang, and Abhik Roychoudhury. 2024. *Fixing Security Vulnerabilities with AI in OSS-Fuzz*. arXiv:2411.03346 [cs] doi:10.48550/arXiv.2411.03346